Zela+ — Automação ESP32 (Sensor de Presença)

Documentação técnica do módulo de hardware que alimenta a detecção de presença/movimento do Zela+. Firmware em `firmware/zela_presenca/zela_presenca.ino`, recebido pelo backend em `POST /ingest/esp32` (`zela/api/ingestao.py`).

1. Visão geral

Um ESP32 lê 2 sensores PIR **SR602** (infravermelho passivo), um na sala e um no quarto, e monitora continuamente se há alguém se movendo. O firmware combina os 2 sensores com um OR lógico — presença é considerada "detectada" se **qualquer um** disparar — e só envia uma leitura ao backend quando esse resultado combinado muda de estado (presente ↔️ ausente), evitando tráfego desnecessário. Essa leitura alimenta a regra de risco do sistema: se não houver nenhuma presença registrada nas últimas 8 horas, o Zela+ classifica o evento como `RISCO` (`zela/domain/monitoramento.py`) e aciona a escada de escalonamento de alerta para a família.

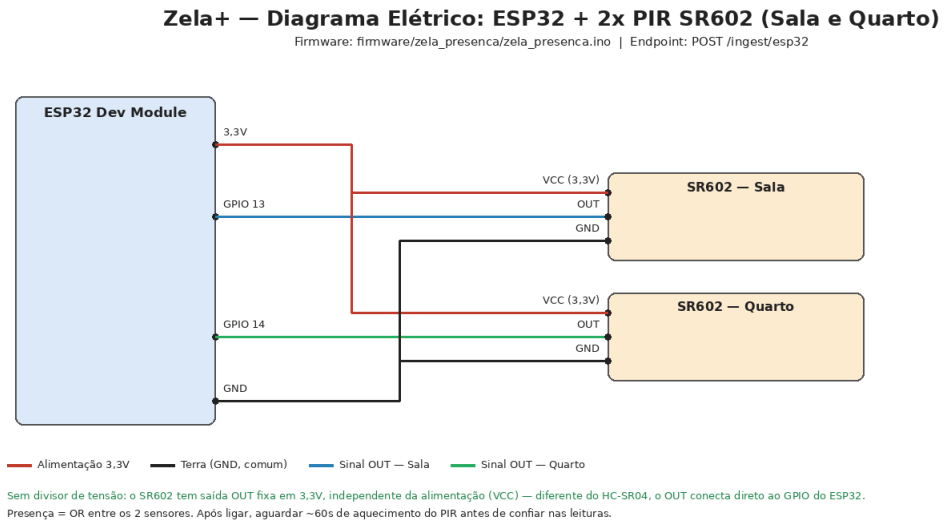
Observação de escopo: o sistema não distingue *em qual* cômodo houve o movimento — o payload enviado ao backend continua sendo um único booleano de presença, combinado dos 2 sensores. Movimento em cômodos sem sensor não é detectado.

2. Lista de materiais (BOM)

Qtd.	Componente	Observação
1	ESP32 Dev Module (ex.: ESP32-WROOM-32 DevKit)	com WiFi integrado
2	Sensor PIR SR602	1 na sala e 1 no quarto; alcance ~5m, alimentação 3,3–15V
—	Jumpers macho-fêmea + protoboard	para montagem sem solda; fiação entre cômodos precisa de cabo mais longo
1	Cabo USB (alimentação/gravação)	conforme o conector da placa (micro-USB ou USB-C)

Nenhum resistor é necessário — ver seção 5.

3. Diagrama elétrico



4. Tabela de pinagem

Sensor	Cômodo	VCC	OUT → ESP32	GND
SR602	Sala	3,3V	GPIO 13	GND
SR602	Quarto	3,3V	GPIO 14	GND

GND é um trilho comum entre o ESP32 e os 2 sensores.

5. Por que não precisa de divisor de tensão (diferente do HC-SR04)

O SR602 tem uma saída `OUT` **fixa em ~3,3V**, independente da tensão de alimentação (`VCC`) — isso é diferente do HC-SR04 (usado numa versão anterior deste módulo), cujo `ECHO` segue a tensão de alimentação (5V) e exigia um divisor resistivo antes do GPIO. Aqui, `OUT` conecta **direto** ao GPIO do ESP32, sem nenhum componente adicional.

O SR602 aceita 3,3V e é alimentado pelo pino `3,3V` do ESP32.

Escolha dos sensores e trabalhos futuros

O MVP usa dois SR602 pela comodidade no desenvolvimento: eles funcionam no pino `3,3V` do ESP32, sem fonte adicional.

Essa escolha não limita o sistema, que aceita modelos diferentes de sensor. O backend recebe apenas a presença combinada, sem depender do modelo. Para incluir um sensor com saída digital de até 3,3V, basta ligá-lo a um GPIO livre e acrescentar o pino em `PINOS_SENSORES`, no firmware.

Trabalho futuro: avaliar o HC-SR505, candidato a melhor desempenho em situações reais, em um ensaio comparativo com o SR602. Ele foi descartado na prototipagem porque pede no mínimo 4,5V e não respondeu quando alimentado pelo pino `5V` (`VIN`) do ESP32. Seu uso prevê alimentação externa de 5V, com GND comum ao ESP32.

6. Lógica do firmware

Resumo de `firmware/zela_presenca/zela_presenca.ino`:

1. No boot: conecta ao WiFi (timeout de 30s) e sincroniza o relógio via NTP (`pool.ntp.org`) — o ESP32 não tem RTC com bateria própria, então sem isso os timestamps enviados seriam inválidos (época Unix).
2. **Aguarda ~60s de aquecimento** (`TEMPO_AQUECIMENTO_MS`) antes de confiar em qualquer leitura — sensores PIR costumam oscilar/disparar falso positivo logo após ligar.
3. A cada 2 segundos (`INTERVALO_LEITURA_MS`), lê os 2 pinos digitais (`digitalRead`) e calcula presença = OR entre os 2.
4. **Só envia ao backend quando o estado combinado muda** (presente ↔️ ausente) — não a cada leitura.
5. Se o WiFi cair, tenta reconectar a cada 15s (`INTERVALO_RECONEXAO_MS`) sem bloquear o loop principal.

7. Configuração de rede e segurança

Antes de gravar o firmware, copiar `firmware/zela_presenca/config.h.example` para `firmware/zela_presenca/config.h` (não versionado — evita vazar credenciais no repositório público) e preencher:

Constante	Valor
`WIFI_SSID` / `WIFI_SENHA`	credenciais da rede WiFi local
`URL_INGESTAO`	endereço do backend — rede local (`http://192.168.x.x:8000/ingest/esp32`) ou VPS pública (`https://seu-dominio.com/ingest/esp32`)
`TOKEN_INGESTAO`	mesmo valor de `ZELA_INGESTAO_TOKEN` no backend — **obrigatório se o backend estiver exposto fora da rede local** (ver `README.md`, seção de configuração)

Quando `TOKEN_INGESTAO` está preenchido, o firmware envia o header `X-Api-Key` em toda requisição; o backend rejeita com `401` qualquer requisição sem o token correto.

8. Fluxo de dados no sistema

```
2x PIR (OR, mudança de estado) -> ESP32 -> POST /ingest/esp32 (X-Api-Key)
  -> LeituraSensor(fonte=ESP32, tipo=PRESENCA) salva no SQLite
ESP32, a cada 5 min -> POST /ingest/esp32 {"tipo": "sinal_de_vida", ...}
  -> LeituraSensor(fonte=ESP32, tipo=SINAL_DE_VIDA), que não conta como movimento
job periódico (a cada 15 min, zela/api/scheduler.py)
  -> classificar_por_regra()
     -> >=8h sem presença com ESP32 vivo, ou ESP32 e pulseira parados -> RISCO
        -> aplicar_escalonamento() -> alerta à idosa e depois à família
     -> só ESP32 (15 min) ou só pulseira (30 min) parados -> AVISO
        -> mensagem única ao primeiro contato familiar, sem escalonamento
```

Cabeamento: use cabo Cat5e até cada sensor. Par laranja: `OUT` + `GND`. Par azul: `VCC` + `GND`.

9. Checklist de montagem e teste

1. Montar o circuito conforme o diagrama da seção 3 (1 sensor na sala e 1 no quarto, fiação de volta até o ESP32).
2. Copiar `config.h.example` para `config.h` e preencher conforme a seção 7.
3. Abrir `firmware/zela_presenca/zela_presenca.ino` no Arduino IDE, placa "ESP32 Dev Module" (pacote "esp32 by Espressif Systems" instalado via Boards Manager).
4. Gravar e abrir o Serial Monitor (115200 bps).
5. Confirmar no log: conexão WiFi, sincronização NTP, e a mensagem de aquecimento dos sensores (~60s).
6. Após o aquecimento, provocar movimento perto de cada um dos 2 sensores e confirmar `POST /ingest/esp32 -> 200` no Serial Monitor.
7. Confirmar que a leitura chegou no backend (painel Streamlit ou consulta direta ao `zela.db`).